

HTTP Methods & Headers

Part A: Methods & the message

#	Do this	How
A1	Audit your verbs	Go through every endpoint from Assignment 4. Confirm the method matches the action — GET reads, POST creates, PUT replaces, PATCH modifies, DELETE removes. Fix any verb that leaked into a URL. Produce the CampusEats method map : a table of action → method + URL.
A2	Model non-CRUD actions	Any action that is not plain create / read / update / delete (cancel, checkout, set Availability..) becomes a POST to a sub-resource, e.g. POST /orders/42/cancel — never POST /cancelOrder.
A3	Mark safe & idempotent	For each endpoint, record two facts: is it safe (changes nothing) and is it idempotent (repeat = same effect)? Confirm no GET ever changes state, and note which endpoints are naturally retry-safe.
A4	Reads take query params	Your list endpoint must filter / sort / paginate through the query string and stay a pure read. (It is a GET — Question 7 asks when POST would instead be justified)
A5	OPTIONS + Allow and override	Implement OPTIONS on one resource, returning an Allow header that lists its methods. If a constrained client cannot send PUT/DELETE, accept X-HTTP-Method-Override — as a documented fallback only.
A6	Show one full exchange	In NOTES.md, paste one complete request (request line + Host + headers + JSON body) and its response (status line + headers + body), each part labelled. Note the HTTP version on the request line.

Part B: Headers on every message

#	Do this	How
B1	Content-Type & negotiation	Every body carries Content-Type: application/json. Honour the Accept header (return JSON), and return 406 Not Acceptable if an unsupported type is demanded. Enable gzip (Content-Encoding) for large JSON if your framework allows.
B2	Correct status + Location	201 + Location on create, 200 on read, 204 on delete, 400 malformed, 404 missing, 409 conflict, 422 domain refusal. Location does double duty — the new resource on 201, and the target on a 3xx redirect (Question 8).
B3	Authorization	Protected endpoints require Authorization: Bearer <token>; a missing or empty token → 401 Unauthorized. Header handling only — no real token system is needed.
B4	Cache a read	Add Cache-Control and an ETag to your single-item GET. The ETag must change when the resource changes — this powers Part C.
B5	Rate-limit signalling	Return X-RateLimit-Limit and X-RateLimit-Remaining, and when a per-client budget is exceeded, 429 Too Many Requests with a Retry-After. The limit is per client, not global.
B6	CORS	Add Access-Control-Allow-Origin and answer the OPTIONS preflight, so a browser page on another origin may call your API (Question 5).

#	Do this	How
B7	Security & general headers	Add X-Content-Type-Options: nosniff and Strict-Transport-Security. Note the Date and Server headers your framework already sends, and that the service is served over HTTPS in production.

Part C: Caching & safe retries

#	Do this	How
C1	Conditional GET → 304	Support If-None-Match on the read from B4: when the client's ETag still matches, return 304 Not Modified with no body.
C2	Conditional write → 412	Support If-Match on your update endpoint: if the ETag no longer matches, return 412 Precondition Failed instead of overwriting — so two editors cannotlobber each other.
C3	Idempotency key	Verify the Idempotency-Key path from Assignment 4 (step C7): the same key returns the original result with no duplicate work. Say where a duplicate would do real damage.
C4	The safe-retry plan	One table for each risky endpoint, which mechanism applies — idempotency key, If-None-Match, or If-Match — and why. Base it on the safe x idempotent split from A3.

Part D: Verify & document

#	Do this	How
D1	curl -v transcript	Capture, with curl -v: a create (201 + Location); the same create repeated with the same Idempotency-Key returning the original; a conditional GET returning 304; a conditional write rejected with 412; and a 400, a 404 and a 401. Failure paths are required.
D2	Headers table	One table for your service: each endpoint × the request headers it needs × the response headers it sets.
D3	Write up NOTES.md	Collect the method map (A1), the headers table (D2), the safe-retry plan (C4), and the eight answers below.

WHAT TO SUBMIT

- ▶ The updated service folder — openapi.yaml (methods, statuses and headers now reflected), source files and tests/ — all running.
- ▶ curl-transcript.txt, run with -v, showing the successful create, the idempotent repeat, the 304 and the 412, and the 400 / 404 / 401 failures.
- ▶ NOTES.md — the CampusEats method map, the headers table, the safe-retry plan, and the eight answers below.
- ▶ Everything committed to the same campuseats repository; submit the link. NOTES.md must show the Team ID and the Roll No and Name of every team member.
- ▶ One Zip file, uploaded by the Team Leader.

NOTES. MD

1. For three of your endpoints, give the method, the success status, and the single response header that matters most — and why.
2. Which of your endpoints are safe, and which are idempotent? Which one is neither, and how did you make it retry-safe?
3. Show one ETag from your service, the request that returns 304, and the write that returns 412. What does each save or prevent?
4. You return 422 for one case and 400 for another. Give the exact request that triggers each, and explain the difference.

5. A browser page on another origin calls your API and is blocked —yet your server logs show a 200. Who blocked it, and which response header fixes it?
6. Name one response where you set Cache-Control to allow caching and one where you must use no-store. Why each?
7. Your search endpoint is a GET. When would POST be the right choice instead, and what do you give up by switching?
8. Location appears on a 201 and on a 3xx. What does it point to in each case?